Section 1 — Incident Header

Incident ID: INC-20260408-008
Date: 08 April 2026
Time Opened (IST): 21:12 IST
Severity: High
Client: TechNova SaaS Startup
Industry: IT / SaaS
L1 Analyst: Priya Nair
L2 Analyst: Rohan Desai
L3 Analyst: Nisha Kapoor
Current Status: Under Investigation
Sentinel Incident Number: 45827

Section 2 — Client & Environment Context

TechNova is a fast-growing SaaS startup headquartered in Bengaluru, with a workforce of approximately 650 employees distributed across India, Singapore, and remote locations globally. Their flagship product is a cloud-based project collaboration platform used by mid-sized enterprises to manage workflows, store documentation, and integrate with third-party services. Because the platform handles client data, internal communications, and API integrations, TechNova sits at a critical intersection of customer trust and operational continuity.

Their infrastructure is heavily cloud-centric. The organization operates on Microsoft 365 E5 licensing, with full deployment of Microsoft Defender for Endpoint across approximately 520 Windows 10/11 endpoints and 35 Azure-hosted Windows Server instances. Identity is fully cloud-managed through Azure AD (Entra ID), with Conditional Access policies enforcing MFA for all users. However, due to developer flexibility requirements, certain engineering endpoints are allowed broader outbound internet access compared to standard corporate devices.

The Azure footprint includes multiple production and staging environments hosted in East US and Central India regions. These environments include Kubernetes clusters, Azure SQL databases containing customer metadata, and storage accounts holding application logs. Continuous integration and deployment pipelines rely on service principals with scoped permissions.

From a network standpoint, TechNova uses a hybrid model — corporate offices connect via high-speed leased lines, while remote developers access resources through a split-tunnel VPN.

This design improves performance but introduces potential blind spots in outbound traffic monitoring.

For this specific incident, one important context point stands out: the engineering team has recently onboarded several third-party libraries and tools for rapid feature development. This has increased outbound traffic patterns from developer machines, making it harder to distinguish legitimate API communication from potential command-and-control activity. This ambiguity becomes critical in interpreting network-based anomalies like the one observed in this incident.

Section 3 — Attack Narrative (Attacker's Perspective)

The operator did not begin with noise. There was no phishing blast, no brute force attempt, no obvious intrusion attempt that would trigger early detection. Instead, the access had already been established days earlier — quietly, through a compromised developer machine that blended into the environment.

The attacker had deployed a lightweight payload designed for persistence and communication, not destruction. It did not write large files to disk or spawn obvious processes. Instead, it embedded itself into a legitimate-looking process context, ensuring it would not stand out in basic endpoint monitoring.

At 17:08 IST, the payload initiated its first outbound connection. It reached out over HTTPS — a deliberate choice. HTTPS traffic would be encrypted, indistinguishable from normal developer API calls, and far less likely to be blocked or inspected deeply.

The destination was a single external IP address hosted on a cloud provider — inexpensive, disposable, and already rotated through multiple campaigns. The payload did not send large volumes of data. Instead, it sent small, consistent packets — just enough to signal presence and await instructions.

The key to its design was timing.

Every 60 seconds — sometimes 58, sometimes 62 — the infected machine would initiate a connection. The slight variation was intentional, just enough to avoid simplistic fixed-interval detection rules, but consistent enough to maintain reliable communication. This pattern continued relentlessly, minute after minute, hour after hour.

Over the next four hours, the attacker observed the environment through these callbacks. Each beacon carried minimal system metadata — hostname, user context, process identifier —

allowing the operator to map the infected endpoint without triggering suspicion through large data transfers.

The attacker was patient. No lateral movement yet. No privilege escalation. No aggressive actions.

Just presence.

At 18:42 IST, the operator issued the first command — a lightweight reconnaissance instruction embedded in the HTTPS response. The payload executed it silently, collecting additional system details and preparing them for the next outbound interval.

Again, no spikes in traffic. No anomalies in volume.

Everything was designed to look like background noise — the kind of repetitive, low-volume traffic that modern SaaS environments generate constantly.

The attacker's objective at this stage was not disruption. It was foothold stabilization and environmental mapping. The longer the beacon remained undetected, the more confident the operator could be in using this endpoint as a launchpad for deeper access — potentially into production systems, API credentials, or internal service communications.

The system was now under observation.

And it had been, quietly, for hours.

Section 4 — Detection & Alert

At 21:12 IST, the alert surfaced quietly — no audible tone across the SOC floor, no red highlight on the central display. It appeared as a **High severity incident** in the Microsoft Sentinel Incidents queue, positioned third from the top under the High severity filter.

Alert Title: *"Possible C2 Beaconing Activity — Consistent HTTPS Interval Detected"*
Severity: High
Entities:

- Host: DEV-LAPTOP-023
- IP Address: 52.184.217.91
- User: dev.rahul@technova.com
  TimeGenerated: 21:11:42 IST

The detection came from a custom Microsoft Sentinel analytics rule designed to identify periodic outbound HTTPS connections — specifically patterns where endpoints communicate with the same external IP at near-regular intervals over an extended duration.

The rule logic was defined as:

```
DeviceNetworkEvents
| where TimeGenerated > ago(4h)  // Look for sustained activity over 4 hours
| where RemotePort == 443  // HTTPS traffic
| summarize ConnectionCount = count(),
        FirstSeen = min(TimeGenerated),
        LastSeen = max(TimeGenerated),
        TimeDeltas = make_list(TimeGenerated)
        by DeviceName, RemoteIP
| where ConnectionCount > 200  // High frequency threshold
| extend DurationMinutes = datetime_diff("minute", LastSeen, FirstSeen)
| where DurationMinutes > 180  // Sustained over at least 3 hours
```

Behind this aggregation, an additional logic layer evaluated the time differences between consecutive connections and flagged patterns where the interval variance remained within a narrow band — in this case between **58 to 62 seconds** across hundreds of events.

For **DEV-LAPTOP-023**, the data showed:

- **Total Connections:** 238 HTTPS requests
- **First Seen:** 17:08 IST
- **Last Seen:** 21:11 IST
- **Average Interval:** ~60.4 seconds
- **Jitter:** ±2 seconds

The payload size for each connection was consistently low — under 1.2 KB — which further contributed to the anomaly profile.

In the Incidents queue, the row displayed a small but telling summary:
"High-frequency outbound HTTPS communication detected — low payload, consistent interval, potential beaconing behavior."

There were no additional alerts tied to this incident — no malware detection, no process alerts, no authentication anomalies.

Just one signal.

And it was entirely network-based.

Section 5 — L1 Triage (Priya Nair)

Priya heard the soft notification tone in her headset before she even looked at the screen.

It wasn't the sharp, urgent tone reserved for Critical alerts — just the standard new-incident chime. Still, her eyes flicked to the top-left corner of her monitor where the Sentinel tab was open in full screen.

21:12 IST.

A new High severity incident had appeared.

She leaned slightly forward, fingers already on the keyboard. Without using the mouse, she navigated directly into the incident using her shortcut keys. The right-hand panel expanded.

"Possible C2 Beaconing… HTTPS interval…" she murmured under her breath, reading the title once, then again slower.

Her eyes dropped to the entity list.

Device: DEV-LAPTOP-023
User: dev.rahul@technova.com
IP: 52.184.217.91

Developer machine.

That immediately shifted how she interpreted the alert.

She tapped into the **Entity Behavior tab**. The screen refreshed, showing the last 24 hours of activity for the device. She scanned quickly — login activity looked normal, no unusual sign-ins, no MFA anomalies.

Then she switched to the **DeviceNetworkEvents preview panel** embedded in the incident.

Rows of outbound connections filled the screen.

Same IP. Same port. Over and over.

Her eyes narrowed.

She clicked one entry. Timestamp: **20:59:02 IST**
Next entry: **21:00:01 IST**

She didn't scroll immediately. Instead, she hovered her finger above the trackpad, thinking.

“Okay… maybe it’s some API polling… Jenkins? GitHub? Internal tool?” she thought.

But then she looked again at the IP.

52.184.217.91

No domain resolution. No recognizable hostname.

She opened a new tab — **AbuseIPDB** — and pasted the IP.

The result loaded in under a second.

Confidence of Abuse: 92%
Reports: Multiple — flagged for botnet and C2 activity.

Priya leaned back slightly.

That was the first moment her posture changed.

She switched back to Sentinel and opened another connection sample. Payload size: 842 bytes. Another: 901 bytes. Another: 776 bytes.

Consistent.

Too consistent.

She hesitated for a second.

“Could this still be a legitimate dev tool… something obscure?” she wondered.

Her fingers tapped lightly on the desk. She had seen noisy developer traffic before — chaotic bursts, inconsistent patterns, API spikes.

But this…

This was rhythm.

She opened the timeline view and scrolled upward.

The pattern stretched back four hours. No breaks. No deviation.

No human behavior looks like this, she thought.

Decision made.

She pressed **Ctrl + Alt + M** — opened the **Microsoft Teams SOC-Incidents channel**. Her cursor blinked in the message box.

She typed deliberately, not rushing.

Escalation Message — 21:18 IST

"@Rohan High severity incident — potential C2 beaconing detected on DEV-LAPTOP-023 (user: dev.rahul@technova.com). Endpoint has made 230+ HTTPS connections to single external IP (52.184.217.91) over ~4 hours with consistent 58–62 sec interval and low payload size. IP flagged on AbuseIPDB (92% malicious confidence). No associated process alerts yet. Pattern appears automated, not user-driven. Requesting deep analysis on source process and confirmation of beacon behavior."

She hit send.

Then added her full triage notes into the Sentinel Incident comments, documenting exactly what she had seen, exactly what she had checked, and exactly why she was escalating.

Only then did she move her hand away from the keyboard.

Section 6 — L2 Investigation (Rohan Desai)

Rohan saw the Teams notification at 21:19 IST.

He didn't open Sentinel immediately.

Instead, he read Priya's message once — then again, slower.

"Consistent 58–62 seconds… 4 hours… no process context yet…"

He leaned back in his chair slightly, eyes still on the message.

Good escalation, he thought. Clean. No assumptions.

He turned to his center monitor and opened the Sentinel incident.

The investigation graph expanded across the screen — a single device node connected to an external IP. No branching. No complexity.

Simple.

Too simple.

He opened **Notepad++** on his second monitor and typed his first question:

What is the exact connection pattern — interval, volume, and consistency?

Then he switched to the KQL workspace.

Query 1 — Validate Beaconing Pattern

```
DeviceNetworkEvents
| where DeviceName == "DEV-LAPTOP-023"  // Focus on affected device
| where RemoteIP == "52.184.217.91"  // Target suspected C2 IP
| where TimeGenerated > ago(5h)  // Expand slightly beyond alert window
| project TimeGenerated
| sort by TimeGenerated asc
| extend PrevTime = prev(TimeGenerated)
| extend IntervalSeconds = datetime_diff("second", TimeGenerated, PrevTime)  // Calculate interval between connections
```

The results streamed in.

Rohan didn't scroll immediately. He looked at the first few rows.

60 sec
59 sec
61 sec
60 sec

He scrolled further.

The pattern held.

He switched to the statistics pane and calculated manually.

Average: ~60.2 seconds
Variance: minimal

He nodded slightly.

"Textbook beacon," he said quietly.

He typed his next question into Notepad++:

What process is generating this traffic?

He pivoted.

Query 2 — Identify Source Process

```
DeviceNetworkEvents
| where DeviceName == "DEV-LAPTOP-023"
```

```
| where RemoteIP == "52.184.217.91"
| where TimeGenerated > ago(5h)
| project TimeGenerated, InitiatingProcessFileName, InitiatingProcessCommandLine, InitiatingProcessSHA256
| summarize count() by InitiatingProcessFileName, InitiatingProcessCommandLine, InitiatingProcessSHA256
```

The output narrowed quickly.

One process dominated all entries:

Process: powershell.exe
Command Line:
powershell.exe -WindowStyle Hidden -ExecutionPolicy Bypass -NoProfile -Command ...

The command line was truncated.

Rohan double-clicked the row to expand it.

Obfuscated script.

He leaned forward.

“Of course,” he muttered.

He copied the command string and pasted it into **CyberChef** in a side browser tab. After a quick decode pass, fragments became visible:

- Invoke-WebRequest
- External IP reference
- Encoded response handling

This wasn’t just beaconing.

It was an active command loop.

He wrote his next question:

Did the attacker issue commands during the beacon window?

He refined the query.

Query 3 — Check Payload Variation (Command Execution Indicator)

```
DeviceNetworkEvents
| where DeviceName == "DEV-LAPTOP-023"
```

```
| where RemoteIP == "52.184.217.91"
| where TimeGenerated > ago(5h)
| summarize AvgBytes = avg(SentBytes), MaxBytes = max(SentBytes), MinBytes = min(SentBytes)
```

Results:

- AvgBytes: 910
- MinBytes: 780
- MaxBytes: 1450

There it was.

A spike.

Rohan scrolled to that timestamp.

18:42 IST

Larger payload.

He cross-referenced with process logs.

At the same timestamp, a PowerShell child execution event appeared — short-lived.

“Command received,” he said.

Now the story was complete.

Attack Timeline

Timestamp (IST)	Event	Data Source	MITRE Technique
17:08	Initial outbound HTTPS beacon begins	DeviceNetworkEvents	T1071.001
17:08–21:11	Consistent beaconing at ~60 sec interval	DeviceNetworkEvents	T1071.001
18:42	Increased payload size indicating command delivery	DeviceNetworkEvents	T1071.001

18:42	PowerShell execution of received instruction	DeviceProcessEvents	T1059.001

IOC Table

IOC	Type	Source	Enrichment Result
52.184.217.91	IP	DeviceNetworkEvents	AbuseIPDB: 92% malicious, associated with botnet infrastructure
powershell.exe (obfuscated command)	Process	DeviceProcessEvents	Suspicious encoded execution pattern
SHA256 (PowerShell process hash)	Hash	DeviceProcessEvents	VirusTotal: flagged by 18 vendors

---

MITRE ATT&CK Mapping

- T1071.001 — Application Layer Protocol: Web Protocols (C2 Beaconing)
- T1059.001 — Command and Scripting Interpreter: PowerShell

Containment Recommendations

Rohan leaned back for a moment before typing these.

Then he wrote them precisely:

1. Isolate **DEV-LAPTOP-023** immediately using Microsoft Defender for Endpoint network isolation.
2. Block IP **52.184.217.91** at perimeter firewall and Azure NSG level.
3. Add IP to **Conditional Access Named Locations — Blocked Entities**.
4. Terminate all active PowerShell processes on affected host via Defender Live Response.
5. Initiate full endpoint scan and memory analysis for persistence mechanisms.
6. Reset credentials for user **dev.rahul@technova.com** and invalidate active sessions.
7. Review outbound traffic logs for other endpoints communicating with same IP.

Rohan added his full investigation notes into Sentinel.

Then he paused for a second before closing the tab.

Four hours of silent presence.

One alert.

Just enough.

Section 7 — L3 Detection Tuning (Nisha Kapoor)

It was 09:40 IST the next morning when Nisha opened the incident report.

Her desk was quiet — no queue, no alerts, just the ultra-wide monitor and Notepad++ already open with yesterday's rule drafts. She clicked into Rohan's investigation document and began reading from the top.

She didn't skim.

Her eyes paused at the timestamps.

17:08 IST — first beacon
21:11 IST — detection fired

Four hours.

She leaned back slightly.

"That's the gap," she said softly.

She minimized the report and opened the **Microsoft Sentinel Analytics Rules editor**. A few clicks later, the rule appeared — *"Possible C2 Beaconing Activity — Consistent HTTPS Interval Detected."*

She read the KQL carefully, line by line.

Before-State Rule (Existing Detection Logic)

```
DeviceNetworkEvents

| where TimeGenerated > ago(4h)  // Look for sustained activity over 4 hours

| where RemotePort == 443  // HTTPS traffic
```

```
| summarize ConnectionCount = count(),
        FirstSeen = min(TimeGenerated),
        LastSeen = max(TimeGenerated),
        TimeDeltas = make_list(TimeGenerated)
        by DeviceName, RemoteIP
| where ConnectionCount > 200  // High frequency threshold
| extend DurationMinutes = datetime_diff("minute", LastSeen, FirstSeen)
| where DurationMinutes > 180  // Sustained over at least 3 hours
```

She didn’t type anything yet.

She just stared at it.

Then she opened Notepad++ and wrote one sentence:

Weakness: The rule requires 3+ hours of activity and high connection volume, delaying detection of early-stage beaconing.

She tapped her pen twice on the desk.

“This didn’t miss the attack,” she said, “it just saw it too late.”

She began typing.

After-State Rule (Improved Detection Logic)

```
DeviceNetworkEvents
| where TimeGenerated > ago(1h)  // Reduce detection window to 1 hour for earlier visibility
| where RemotePort == 443  // Focus on HTTPS traffic
| summarize ConnectionCount = count(),
        FirstSeen = min(TimeGenerated),
        LastSeen = max(TimeGenerated),
        TimeList = make_list(TimeGenerated)
        by DeviceName, RemoteIP
| where ConnectionCount > 40  // Lower threshold to detect earlier beacon cycles
| extend DurationMinutes = datetime_diff("minute", LastSeen, FirstSeen)
| where DurationMinutes > 30  // Detect sustained behavior over 30 minutes instead of 3 hours
| mv-expand TimeList  // Expand timestamps for interval calculation
| sort by DeviceName, RemoteIP, TimeList asc
```

```
| extend PrevTime = prev(TimeList)
| extend IntervalSeconds = datetime_diff("second", TimeList, PrevTime)
| where IntervalSeconds between (50 .. 70)  // Identify consistent beacon interval range
| summarize BeaconCount = count(),
        AvgInterval = avg(IntervalSeconds),
        MinInterval = min(IntervalSeconds),
        MaxInterval = max(IntervalSeconds)
        by DeviceName, RemoteIP
| where BeaconCount > 20  // Ensure repeated pattern consistency
```

She stopped typing and read it back once.

Then again.

This time, she imagined it against yesterday's data.

Instead of firing at 21:11 IST…

This would have triggered around **17:45 IST**.

Almost three and a half hours earlier.

She nodded once.

"This catches behavior, not volume," she said.

In her notes, she wrote:

The improvement shifts detection from late-stage aggregation (high volume over long duration) to early behavioral consistency (interval-based beaconing). By lowering thresholds and explicitly calculating interval regularity, the rule detects command-and-control activity within the first operational phase instead of after prolonged dwell time.

She saved the draft.

Then opened the MITRE Navigator tab briefly — confirmed mapping.

T1071.001 still correct.

She closed it.

No more changes needed.

Section 8 — Final Incident Report

Part A — Executive Summary

A compromised developer workstation within TechNova's environment was found communicating with an external malicious server over several hours. The communication occurred in a repetitive pattern, indicating unauthorized remote control activity. The issue was detected by internal monitoring systems and quickly investigated.

The affected system was isolated before any significant damage or data loss occurred. No evidence of lateral movement or impact to production systems was found.

Immediate containment actions were taken, and additional safeguards have been implemented to improve early detection of similar behavior in the future.

Part B — Full Technical Summary

Incident ID: INC-20260408-008
Client: TechNova SaaS Startup
Attack Type: Command and Control (C2 Beaconing)

MITRE ATT&CK Techniques

- T1071.001 — Application Layer Protocol: Web Protocols
- T1059.001 — Command and Scripting Interpreter: PowerShell

Timeline Summary

Timestamp (IST)	Event
17:08	Initial outbound HTTPS beacon initiated from DEV-LAPTOP-023
17:08–21:11	Continuous beaconing at ~60-second intervals
18:42	Increased payload size indicating command delivery
18:42	PowerShell execution triggered via C2 response
21:11	Sentinel detection rule triggered
21:18	L1 escalation to L2
21:25–22:10	L2 investigation completed

22:15	Endpoint isolated and containment initiated

Impact Assessment

- No confirmed data exfiltration
- No lateral movement observed
- No access to production systems detected
- Infection contained to a single developer endpoint

IOC List

IOC	Type	Source	Enrichment Result
52.184.217.91	IP Address	DeviceNetworkEvents	AbuseIPDB: 92% malicious, linked to botnet infrastructure
powershell.exe (encoded command)	Process	DeviceProcessEvents	Suspicious obfuscated execution pattern
SHA256 Hash (PowerShell instance)	File Hash	DeviceProcessEvents	VirusTotal: flagged by 18 security vendors

---

Containment Actions Taken

- Endpoint **DEV-LAPTOP-023** isolated via Microsoft Defender for Endpoint
- External IP **52.184.217.91** blocked at firewall and Azure NSG
- IP added to Conditional Access blocked locations list
- Malicious PowerShell processes terminated
- User credentials reset and active sessions revoked
- Full endpoint scan and forensic review initiated

Lessons Learned

- Existing detection logic identified beaconing behavior too late in the attack lifecycle

- Developer endpoints with high outbound traffic require behavior-based monitoring, not just volume-based thresholds
- Low-volume, high-frequency communication patterns can evade traditional detection logic

---

Action Items

Action	Owner	Priority	Target Date
Deploy updated beaconing detection rule in Microsoft Sentinel	Nisha Kapoor (L3)	High	10 April 2026
Conduct threat hunt for similar beaconing patterns across all endpoints	Rohan Desai (L2)	High	11 April 2026
Review outbound network policies for developer endpoints	TechNova IT Security Team	Medium	15 April 2026
Implement alerting for repeated low-payload HTTPS traffic patterns	Nisha Kapoor (L3)	High	12 April 2026
Conduct developer awareness session on endpoint security risks	TechNova Security Awareness Team	Medium	20 April 2026